

QA 사전과제 답안

문제1: 테스트 자동화 전략 설계 (서술)

시나리오

- 배달의민족 앱에 새로운 "실시간 주문 현황" 화면을 추가하는 스크린트가 시작됩니다. 개발자 2명이 이번 주부터 개발을 시작했으며, 오픈은 2주 후입니다. 현재 이 서비스에는 자동화 테스트가 전혀 없습니다.

1-1. 자동화 전략 수립

이 기능에 대해 무엇을 자동화하고 무엇을 수동으로 남길지, 판단 기준과 함께 작성해주세요.

- UI / API / E2E 각 레이어에서 자동화 범위를 어떻게 배분할지(테스트 피라미드 관점) 설계해주세요.
- 자동화 투자 대비 효과(ROI)를 어떤 기준으로 판단할 것인지 설명해주세요.

1-1 답변

Q1. 이 기능에서 먼저 확인할 문제는 무엇인가?

같은 주문의 상태가 사장님, 라이더, 고객 화면에 맞게 표시되는지 먼저 확인하겠습니다. 서버에서 취소된 주문이 고객 화면에는 '조리 중'으로 남거나, 사장님과 라이더에게 취소 안내가 전달되지 않는 경우가 대표적입니다.

자동화할 항목은 고객에게 미치는 영향, 반복 횟수, 코드나 로그로 결과를 판정할 수 있는지를 기준으로 정하겠습니다.

- 권한:** 고객은 본인 주문만, 사장님은 매장 주문만, 라이더는 담당 배달만 조회하고 처리할 수 있어야 합니다. 화면에서 버튼을 숨겨도 권한 없는 API 요청은 서버에서 차단해야 합니다.
- 접수와 취소:** 사장님의 접수와 고객센터의 취소 요청이 겹칠 때 처리 순서와 최종 상태를 개발 전에 정하겠습니다. 취소되면 주문, 결제, 배달 상태와 각 앱의 안내가 정책에 맞게 바뀌는지 확인합니다.
- 상태 갱신:** 반영 허용 시간을 정하고 이벤트의 중복, 누락, 역순 도착을 검사합니다. 늦게 도착한 '배달 시작' 이벤트가 '배달 완료'를 되돌려서는 안 됩니다.
- 재접속:** 네트워크 재연결이나 앱 재실행 후 최신 주문 상태를 가져오는지 확인합니다.

문구의 이해도, 사용성, 접근성 등 사람이 판단해야 하는 부분은 실제 기기에서 수동으로 확인하고 탐색적 테스트를 병행하겠습니다.

Q2. UI, API, E2E의 자동화 범위를 어떻게 나눌 것인가?

반복 실행이 빠르고 실패 원인을 찾기 쉬운 API 테스트를 가장 많이 구성하겠습니다. UI는 화면 상태를, E2E는 서비스가 연결되는 주문 흐름을 확인하는 데 사용하겠습니다.

영역	자동화할 내용	선정 이유
API	주문 상태 변경, 권한, 취소, 중복 요청, 잘못된 처리 순서, 실패 시 상태 유지	여러 조건을 빠르게 반복 검증할 수 있음
UI	상태 문구, 버튼 노출과 활성화, 로딩, 오류, 재연결 후 표시	서버 처리 결과가 화면에 맞게 전달되는지 확인
E2E	사장님 접수 → 라이더 진행 → 고객 화면 반영, 주문 취소	실제 연동을 확인하되 실행 시간과 유지보수 비용을 고려해 핵심 흐름만 구성
수동	문구 이해도, 접근성, 사용성, 예상하지 못한 상태 조합	실제 사용 과정에서 생기는 혼란과 불편을 확인

Q3. 자동화 투자 효과는 어떻게 판단할 것인가?

구축과 운영에 든 비용을 반복 실행으로 절약한 시간과 비교하겠습니다. 비용에는 구현, 환경 구성, 유지보수, 실패 분석, 결과 확인과 인프라를 포함합니다. 효과는 수동 시험 시간 절감과 결함을 일찍 발견해 줄인 수정 비용으로 보겠습니다.

다만 권한 오류나 잘못된 주문 처리처럼 고객 피해가 큰 항목은 시간 절감이 작아도 우선 자동화하겠습니다.

Q4. 개발자 2명과 2주라는 조건에서는 어떻게 진행할 것인가?

요구사항 조율부터 재검증과 배포 일정 관리까지 맡았던 경험을 바탕으로, 개발과 검증을 병행하겠습니다.

- 개발 전:** 상태 전이, 권한, 취소 정책과 API 명세를 확인하고 테스트 데이터를 준비합니다.
- 개발 중:** 구현된 API부터 검증하고, 변경사항이 기존 기능에 미치는 영향을 확인해 시험 순서를 조정합니다.
- 출시 전:** 핵심 주문 흐름을 재검증하고 미검증 항목, 남은 결함과 대응 방안을 보고합니다.

일정이 부족하면 영향이 작은 표시 차이나 일부 기기 조합을 후속 검증으로 미루겠습니다. 권한이나 주문 처리에 문제가 남으면 기능 제한, 기존 화면 유지, 제한 공개, 일정 조정 중 가능한 방안을 제시하겠습니다.

1-2. 파이프라인 통합

작성한 자동화 테스트를 CI/CD 파이프라인의 어느 단계에서 어떻게 실행할지 설계해주세요.

- Flaky 테스트가 발생했을 때의 대응 정책(격리, 재시도, 원인 분석)을 함께 서술해주세요.

1-2 답변

Q1. 자동화 테스트를 언제 실행할 것인가?

1-1에서 정한 API, UI, E2E 테스트를 실행 시간과 검증 목적에 따라 CI/CD 단계에 배치하겠습니다.

시점	실행할 테스트
PR 검증 또는 병합 전	테스트 서버나 스텝에서 API 규칙과 계약 검증
테스트 환경 배포 후	실제 서버와의 API 연동 검증, 주요 UI 테스트
주요 기능 연결 후	접수부터 배달 진행까지의 상태 반영, 취소 등 핵심 E2E
출시 전	핵심 API, UI, E2E 회귀 테스트
운영 배포 후	실제 주문을 생성하거나 취소하지 않는 조희 중심의 점검

실행 시간이 긴 상태 조합이나 여러 기기에서의 테스트는 별도로 실행해 코드 변경 시 피드백이 늦어지지 않게 하겠습니다.

Q2. 테스트가 실패하면 무엇부터 확인할 것인가?

실패 시 주문 ID, 요청 ID, 빌드, 계정, 테스트 데이터, 화면과 로그를 남기겠습니다. 같은 주문의 요청, 권한 확인, 상태 저장, 이벤트 전달, 화면 표시 순서로 추적해 처음 어긋난 지점을 찾겠습니다.

안전운전 자동시험에서 시뮬레이터 로딩 문제와 GPS 재생 문제를 나눠 원인을 찾았던 것처럼, 제품 결함과 자동화 코드, 데이터, 환경 문제를 구분하겠습니다.

Q3. Flaky 테스트는 어떻게 처리할 것인가?

최초 실패 기록을 보존한 뒤 원인을 조사하겠습니다.

- ↳ **재시도:** 기존 주문과 데이터를 확인한 뒤 진단 목적으로 한 번 실행합니다. 처리 여부를 모르는 결제는 다시 시도하지 않습니다.
- ↳ **격리:** 자동화 자체의 문제로 확인되면 별도 실행으로 분리하고 담당자와 수정 기한을 정합니다.
- ↳ **대체 검증:** 중요한 기능은 격리 기간에 API 테스트나 수동 시험으로 확인합니다.
- ↳ **복귀:** 원인을 수정하고 최초 실패 조건과 관련 회귀 테스트를 통과하면 파이프라인에 다시 포함합니다.

문제2: B마트 재고 API 테스트 자동화 구현 (코드 필수)

배경

- └ B마트는 배달의민족이 운영하는 장보기 서비스입니다.
- └ 실시간 재고 관리가 핵심이며, 피크 타임에는 동일 상품에 대한 동시 주문이 빈번하게 발생합니다.
- └ 재고 차감과 주문 처리 사이의 데이터 정합성이 서비스 신뢰도와 직결됩니다.

재고 처리 규칙

이벤트	재고 변동	고객 화면 처리	비고
고객이 장바구니에 상품 추가	변동 없음 (조회만)	현재 재고 수량 노출	장바구니 담는 시점에는 재고 차감 없음
주문 결제 완료	주문 수량만큼 즉시 차감	주문 완료 화면	동시 주문 충돌 시 선착순 처리. 이후 요청은 품절 처리
주문 취소 (결제 완료 후)	차감된 재고 즉시 복구	취소 완료 알림	복구 후 다른 고객이 즉시 구매 가능 상태 전환
재고 차감 후 수량 = 0	해당 상품 품절 전환	"품절" 배지 표시, 주문 버튼 비활성화	다른 탭에서 조회 중인 고객에게도 즉시 반영되어야 함
운영자가 재고 수동 수정	입력값으로 즉시 반영	실시간 업데이트	수정 중 주문 발생 시 처리 순서 정의 필요

동시성 시나리오

- └ 특정 인기 상품의 재고가 마지막 1개 남았습니다. 이때 고객 A와 고객 B가 거의 동시에 해당 상품을 결제합니다. 두 결제 요청이 서버에 수십 밀리초 간격으로 도달합니다

테스트 케이스 설계

- └ 아래 가상의 API가 있다고 가정하고(명세는 지원자가 합리적으로 정의), 테스트 자동화 코드를 작성해주세요.
 - └ GET /v1/products/{productId}/stock — 재고 조회
 - └ POST /v1/orders — 주문 생성(재고 차감)
 - └ POST /v1/orders/{orderId}/cancel — 주문 취소(재고 복구)

요구사항

- └ 정상 케이스, 재고 복구, 품질 전환을 포함한 API 자동화 테스트 코드 10개 이상 작성
- └ 재고 1개 남은 상품에 두 요청이 수십 ms 간격으로 도달하는 동시성 테스트를 코드로 구현 (병렬 요청 → 1건 성공/1건 품질 검증)
- └ 실제 서버가 없으므로 Mock 서버(WireMock, responses 등) 또는 직접 구현한 간단한 스텝 서버를 사용해 실행 가능한 상태로 제출
- └ 테스트 코드의 구조 설계(fixture, 데이터 준비, 검증 헬퍼 등)에 대한 설명을 README에 포함

문제 2 답변

Q1. 어떤 재고 처리 규칙을 검증했는가?

결제 완료 주문은 수량만큼 재고를 차감하고, 취소는 차감했던 수량만 복구하도록 구현했습니다. 장바구니 추가와 조회는 재고를 바꾸지 않으며, 재고가 0이면 품질로 처리합니다.

- └ **주문:** 마지막 1개에 요청이 겹쳐도 한 건만 성공해야 합니다.
- └ **취소:** 같은 취소 요청이 반복되거나 동시에 들어와도 재고는 한 번만 복구해야 합니다.
- └ **실패:** 품질이나 잘못된 요청은 재고를 바꾸지 않아야 합니다.
- └ **품질 해제:** 취소로 재고가 복구되면 다른 고객이 다시 주문할 수 있어야 합니다.

응답 코드와 함께 초기 재고 - 성공 주문 수량 + 취소로 복구한 수량 = 현재 재고를 확인하고, 저장된 주문 정보도 비교했습니다.

API 응답은 조회 200, 주문 성공 201, 품질 409, 취소 200으로 정의했습니다. 없는 상품이나 주문은 404, 잘못된 수량은 400입니다. 운영자 수동 수정은 주문과의 처리 순서를 정해야 하지만, 관리자 API가 없어 이번 구현에서는 제외했습니다.

Q2. 어떤 테스트 케이스를 작성했는가?

구분	검증 내용
주문	1개, 복수, 전량 주문과 품질, 재고 초과, 잘못된 수량
취소	수량 복구, 품질 해제, 다른 고객의 재구매, 반복 취소와 병렬 취소
수량 정합성	반복 조회, 없는 상품이나 주문, 상품 간 영향, 여러 주문과 취소 후 최종 수량
동시 주문	재고 1개에 두 요청이 도착했을 때 처리 중첩, 실제 접수 간격과 선착순 결과

고유 TC 18개를 작성했고, 매개변수를 포함한 기본 실행은 25건입니다. 동시 주문 TC-CON-001의 50회 반복은 별도 실행입니다. 테스트 케이스 문서에서 확인할 수 있습니다.

Q3. 수십 ms 간격의 동시 주문은 어떻게 구현했는가?

첫 요청이 처리되는 동안 두 번째 요청이 서버에 도착하도록 구성했습니다.

1. 매 회차 상품 재고를 1개로 초기화하고 고객 A와 B의 독립된 주문 요청을 준비합니다.
2. 먼저 요청할 고객과 10~50ms의 전송 간격을 무작위로 선택합니다.
3. 첫 요청이 서버에 접수된 시점부터 선택한 간격을 기다린 뒤 두 번째 요청을 보냅니다.
4. 요청 중첩을 재현하기 위한 테스트용 동기화 지점에서 첫 요청을 재고 처리 직전에 기다리게 합니다. 두 요청이 모두 도착하면 대기를 풀고 재고 로직을 실행합니다.
5. 스텝 서버가 접수 번호를 부여한 순서대로 처리합니다. 먼저 접수된 주문 한 건은 성공하고 다른 주문은 품질인지 확인합니다. 최종 재고는 0, 저장된 주문은 한 건이어야 합니다. 실패 접수 간격은 설정값 이상 100ms 미만이어야 통과합니다.

기존 50회 시험은 모두 통과했고, 서버 접수 간격은 **31.226~79.540ms**였습니다. 요청 순서, 설정 간격, 실패 간격과 결과는 시험성적서에 기록했습니다. 조건을 벗어난 실행은 재시도하지 않고 실패로 남깁니다.

기본 API 시험은 run_tests.bat, 동시 주문 반복 시험은 run_concurrency.bat로 실행합니다. 서버 구성과 fixture, 데이터 준비, 검증 함수는 README에 설명했습니다.

문제3: E2E 자동화 코드 리뷰 및 리팩토링 (코드 필수)

└ 아래는 주문 플로우를 검증하는 E2E 자동화 테스트 코드(Kotlin + UIAutomator)입니다.

```
@Test
fun testOrderFlow() {
    val shopItem = device.wait(Until.findObject(By.textContains("치킨")), 5000)
    shopItem.click()
    val menuItem = device.wait(Until.findObject(By.textContains("후라이드")), 3000)
    menuItem.click()
    val orderBtn = device.wait(Until.findObject(By.textContains("주문하기")), 3000)
    orderBtn.click()
    val payBtn = device.wait(Until.findObject(By.textContains("결제")), 3000)
    payBtn.click()
    Thread.sleep(5000)
    val statusText = device.wait(Until.findObject(By.textContains("접수 대기")), 3000)
    Assert.assertNotNull(statusText)
}
```

요구사항

- └ 실제 운영 환경에서 발생할 수 있는 문제점을 모두 찾아 설명해주세요. (안정성, 유지보수성, 신뢰성 관점)
- └ 직접 리팩토링한 코드를 제출해주세요. Page Object 등 설계 패턴 적용, 대기 전략 개선, 검증 강화를 포함해야 합니다. (실행 환경이 없어도 무방하며, 설계 의도가 드러나는 완성된 코드 형태면 충분)
- └ 이 테스트가 CI에서 간헐적으로 실패한다면(Flaky) 어떤 순서로 원인을 추적할지 서술해주세요.

문제 3 답변

Q1. 기존 코드에는 어떤 문제가 있는가?

화면 문구에 의존하고, 상태와 무관한 고정 대기과 분산된 Timeout 설정으로 정상 기능도 CI에서 간헐적으로 실패할 수 있습니다. 마지막에는 '접수 대기' 문구의 존재만 확인하므로 실제 주문 성공 여부도 충분히 검증하지 못합니다.

1. 안정성

- ㄴ 고정 대기: `Thread.sleep(5000)`은 화면 준비 여부와 관계없이 항상 5초를 기다립니다. 화면이 빨리 준비되면 불필요한 대기 시간이 발생하고, 5초 이후에도 준비되지 않아 다음 단계의 최대 대기 시간까지 넘으면 실패할 수 있습니다. 특정 화면이나 상태가 나타나는 즉시 진행하는 조건 기반 대기로 변경하겠습니다.
- ㄴ Timeout 관리: `device.wait()`는 최대 대기 시간 안에 요소가 나타나면 즉시 반환합니다. 3000ms, 5000ms는 고정 수면 시간이 아니라 최대 대기 시간입니다. 단계별 Timeout이 코드에 직접 지정되어 있어 실행 환경 변화에 대응하기 어렵습니다. 공통 Timeout 정책을 두고 실제 지연 수준과 서비스 허용 시간에 맞게 관리하겠습니다.
- ㄴ 화면 전환 확인 부족: 클릭 후 다음 요소를 기다리지만, 의도한 화면으로 이동했는지는 따로 확인하지 않습니다. 전환이 끝나지 않았거나 다른 화면으로 이동했을 때도 다음 요소를 찾으려 할 수 있습니다.
- ㄴ 조회 실패와 버튼 상태 처리 부족: `device.wait(Until.findObject(...), timeout)`이 제한 시간 안에 요소를 찾지 못하면 null을 반환할 수 있습니다. 이를 확인하지 않고 바로 `click()`하면 null 관련 예외로 끝나 어느 대상을 기다리다 실패했는지 알기 어렵습니다. 대상과 최대 대기 시간을 포함한 실패 메시지가 필요합니다. 비활성 버튼, 팝업에 가려진 요소나 교체된 뷰도 확인해야 합니다.
- ㄴ 테스트 데이터 의존: 가게가 영업을 종료했거나 메뉴가 품질이면 테스트가 실패합니다. 사용할 수 있는 전용 매장, 메뉴와 계정을 준비하고 로그인, 배송지, 장바구니 상태를 고정해야 합니다.

2. 유지보수성

- ㄴ 화면 문구에 직접 의존: `By.textContains("주문하기")`는 버튼 문구가 '주문'으로 바뀌면 실패합니다. 개발팀과 합의한 resource-id를 우선 사용하고, `contentDescription`을 쓴다면 문구나 언어 변경의 영향도 확인해야 합니다.
- ㄴ 부분 문구의 중복: '치킨'이 가게 이름, 메뉴 설명과 추천 영역에 함께 있으면 의도하지 않은 요소를 선택할 수 있습니다. 식별자와 정확한 테스트 이름을 함께 사용해야 합니다.
- ㄴ 한 함수에 몰린 화면 로직: 매장 선택, 메뉴 선택, 주문, 결제와 검증이 섞여 있어 UI 변경 시 여러 테스트를 수정해야 합니다. `ShopPage`, `MenuPage`, `PaymentPage`처럼 화면별 Page Object로 분리하는 것이 적합합니다.
- ㄴ 분산된 Timeout: 여러 곳의 값을 따로 바꾸면 변경이 누락되기 쉽습니다. 공통 설정으로 관리해야 합니다.
- ㄴ 재사용할 함수 부족: 버튼 대기과 클릭, 화면 전환 확인을 다른 E2E에서도 다시 작성해야 합니다. 공통 동작을 함수로 분리해야 합니다.

3. 신뢰성

`Assert.assertNotNull(statusText)`는 화면에 '접수 대기'가 있으면 통과합니다. 다음 경우를 구분하지 못합니다.

- ㄴ 이전 주문이나 다른 주문의 상태가 표시된 경우
- ㄴ 잘못된 메뉴, 수량이나 금액으로 주문한 경우
- ㄴ 결제는 실패했지만 다른 영역에 같은 문구가 있는 경우
- ㄴ 화면에는 결과가 표시됐지만 서버에는 주문이 저장되지 않은 경우

주문번호, 주문한 메뉴, 결제 결과와 현재 상태가 이번 주문에 해당하는지 확인해야 합니다. API나 서버 로그를 사용할 수 있다면 주문 ID로 서버 결과와 화면을 대조해야 합니다. 정상 주문이 빠르게 다음 상태로 넘어가 '접수 대기'를 놓칠 수 있으므로, 해당 상태를 유지할 테스트 조건이나 허용 상태 전이도 정해야 합니다. 결제 후 응답만 유실된 상황에서 전체 흐름을 재실행하면 중복 주문이 생길 수 있습니다. 실패 당시 화면, 로그와 주문 ID를 남기고 서버 처리 여부부터 확인해야 합니다.

Q2. 어떤 부분을 리팩토링했는가?

OrderFlowTest.kt에 화면별 Page Object, 공통 대기와 실패 자료 저장 코드를 한 파일로 작성했습니다.

```
class OrderFlowRefactoredTest(
    private val device: UiDevice,
    private val packageName: String,
    private val artifactDir: File
){
    fun testOrderFlow() {
        runWithFailureArtifacts {
            ShopPage().selectShop("치킨")
                .selectMenu("후라이드")
                .proceedToPayment()
                .pay()
                .verifyOrder(expectedStatus = "접수 대기", expectedMenu = "후라이드")
        }
    }

    // 최대 timeout까지 찾되 발견 즉시 반환. 못 찾으면 대상과 제한 시간을 명시해 실패.
    private inner class UiWait {
        private val defaultTimeout = 10_000L

        fun visible(selector: BySelector, description: String, timeout: Long = defaultTimeout): UiObject2 {
            return device.wait(Until.findObject(selector), timeout)
                ?: throw AssertionError("$description 을(를) ${timeout}ms 안에 찾지 못했습니다.")
        }

        fun click(selector: BySelector, description: String) {
            // 표시된 버튼이 활성화되고 클릭 가능한 상태가 될 때까지 기다림.
            val ready = By.copy(selector).enabled(true).clickable(true)
            val element = visible(ready, "$description 클릭 가능 상태")
            element.click() // 결제 중복을 막기 위해 클릭 자체는 재시도하지 않음.
        }
    }

    // 화면별 조작을 분리하고, ID와 정확한 이름으로 대상을 선택.
    private inner class ShopPage {
        private val wait = UiWait()

        fun selectShop(shopName: String): MenuPage {
            wait.click(By.res(packageName, "shop_name").text(shopName), "매장 '$shopName'")
            return MenuPage().waitUntilDisplayed()
        }
    }
}
```

```
private inner class MenuPage {
    private val wait = UiWait()
    fun waitUntilDisplayed(): MenuPage {
        wait.visible(By.res(packageName, "menu_list"), "메뉴 목록")
        return this
    }
    fun selectMenu(menuName: String): OrderPage {
        wait.click(By.res(packageName, "menu_name").text(menuName), "메뉴 '$menuName'")
        return OrderPage().waitUntilDisplayed()
    }
}
}
```

```
private inner class OrderPage {
    private val wait = UiWait()
    fun waitUntilDisplayed(): OrderPage {
        wait.visible(By.res(packageName, "order_button"), "주문하기 버튼")
        return this
    }
    fun proceedToPayment(): PaymentPage {
        wait.click(By.res(packageName, "order_button"), "주문하기 버튼")
        return PaymentPage().waitUntilDisplayed()
    }
}
}
```

```
private inner class PaymentPage {
    private val wait = UiWait()
    fun waitUntilDisplayed(): PaymentPage {
        wait.visible(By.res(packageName, "payment_button"), "결제 버튼")
        return this
    }
    fun pay(): OrderStatusPage {
        wait.click(By.res(packageName, "payment_button"), "결제 버튼")
        return OrderStatusPage().waitUntilDisplayed()
    }
}
}
```

// 상태 문구만 확인하지 않고 주문 메뉴와 주문번호도 검증.

```
private inner class OrderStatusPage {
    private val wait = UiWait()
    fun waitUntilDisplayed(): OrderStatusPage {
        wait.visible(By.res(packageName, "order_status"), "주문 상태")
        return this
    }
}
```

```

    }
    fun verifyOrder(expectedStatus: String, expectedMenu: String) {
        // 요소만 먼저 표시되는 경우를 고려해 기대 문구와 주문번호가 준비될 때까지 대기.
        val status = wait.visible(By.res(packageName, "order_status").text(expectedStatus), "주문 상태: $expectedStatus").text
        val menu = wait.visible(By.res(packageName, "ordered_menu").text(expectedMenu), "주문 메뉴: $expectedMenu").text
        val orderNumber = wait.visible(By.res(packageName, "order_number").text(java.util.regex.Pattern.compile(".*\\S.*", java.util.regex.Pattern.DOTALL)), "값이 있는 주문 번호").text
        assertEquals("주문 상태가 예상과 다릅니다.", expectedStatus, status)
        assertEquals("주문 메뉴가 예상과 다릅니다.", expectedMenu, menu)
        assertTrue("주문 번호가 생성되지 않았습니다.", !orderNumber.isNullOrBlank())
    }
}

// 실패 당시 화면과 UI 구조를 저장해 CI 원인 분석에 사용.
private fun runWithFailureArtifacts(block: () -> Unit) {
    try {
        block()
    } catch (e: Throwable) {
        saveFailureArtifacts()
        throw e
    }
}

private fun saveFailureArtifacts() {
    val directory = artifactDir
    runCatching { directory.mkdirs() }
    val timestamp = System.currentTimeMillis()
    runCatching { device.takeScreenshot(File(directory, "order_flow_$timestamp.png")) }
    runCatching { device.dumpWindowHierarchy(File(directory, "order_flow_$timestamp.xml")) }
}
}

```

설계 의도

화면 탐색과 조작 방법은 Page Object에 두고 테스트에서는 사용자 흐름만 표현했습니다. 버튼 ID나 화면 구조가 변경되어도 해당 Page를 수정하면 다른 테스트에 미치는 영향을 줄일 수 있습니다. 각 Page는 다음 화면의 요소를 확인한 뒤 반환합니다. 공통 UiWait는 device.wait(Until.findObject(...), timeout)으로 조건을 만족하는 요소를 찾으면 즉시 반환합니다. 클릭할 때는 enabled(true)와 clickable(true)를 함께 지정해 버튼이 활성화될 때까지 기다립니다. 제한 시간 안에 찾지 못하면 ?: throw AssertionError(...)로 대상과 최대 대기 시간을 명시합니다. 결과 화면에서는 기대 상태와 메뉴, 비어 있지 않은 주문번호까지 기다립니다. 클릭 자체는 재시도하지 않습니다. 부분 문구 검색은 resource-id와 정확한 이름의 조합으로 바꿨고, 실패하면 화면 캡처와 UI hierarchy 저장을 시도합니다.

적용 전제와 검증 범위

- ㄴ CI에서는 실행 전 로그인, 배송지, 장바구니, 시작 화면과 테스트 데이터를 초기화합니다.
- ㄴ 이번 코드는 UI의 주문번호, 메뉴와 상태를 검증합니다. 신규 주문 생성, 금액과 결제 성공 여부는 실제 연동 환경에서 주문 ID로 서버 결과와 대조합니다.
- ㄴ 실제 앱 정보가 없어 resource-id는 예시입니다. 코드의 ‘치킨’과 ‘후라이드’는 정확히 일치하는 테스트 매장명과 메뉴명입니다. 실제 적용 시 개발팀과 합의한 식별자와 테스트 데이터로 교체합니다.
- ㄴ 버튼의 활성화와 클릭 가능 상태까지 기다리지만 팝업에 가리지 않았는지, 터치가 처리됐는지까지 보장하지는 않습니다. 다음 화면을 확인하며, 중복 요소 검사와 stale 복구는 별도 보완 사항입니다.
- ㄴ 실패 시 PNG와 XML 저장을 시도합니다. CI에서는 앱이 제거되기 전에 파일을 가져오고, logcat, 빌드, 기기와 테스트 데이터 정보를 함께 보관해야 합니다.

Q3. CI에서 간헐적으로 실패하면 어떤 순서로 원인을 추적할 것인가?

최초 실패 정보를 보존한 뒤 제품, 테스트 코드, 데이터와 환경 중 어디서 문제가 발생했는지 구분하겠습니다.

1. 실패 정보 확보: 실패 시점의 화면 캡처, UI hierarchy, 로그, 빌드, 기기, 테스트 데이터와 실패 단계를 저장합니다.
2. 실패 지점 확인: 매장 선택, 메뉴 선택, 주문, 결제, 주문 상태 확인 중 처음 문제가 발생한 단계를 찾습니다. 결제 후 실패했다면 서버 주문과 결제 내역부터 확인하고, 처리 여부가 불명확하면 재결제하지 않습니다.
3. 원인 분리: 기존 주문을 확인하고 데이터를 정리한 뒤 동일 조건으로 진단 목적의 재실행을 한 번 수행합니다. 성공한 실행과 비교해 UI 로딩 지연, 선택자, 네트워크, 데이터와 서버 처리 문제를 구분합니다.
4. 재현 조건 축소: 특정 기기, 빌드, 네트워크나 데이터에서만 발생하는지 조건을 하나씩 바꿉니다. 단독 실행만 통과한다면 실행 순서와 병렬 실행의 데이터 공유도 확인합니다.
5. 수정 후 회귀: 원인을 수정한 뒤 최초 실패 조건과 관련 주문 흐름을 다시 실행해 재발 여부를 확인합니다.

재실행에 성공해도 최초 실패 기록은 삭제하지 않습니다. 테스트 코드나 환경 문제로 확인된 항목은 담당자, 수정 기한과 대체 검증을 정해 격리하겠습니다. 실제 사용자에게도 발생할 수 있는 주문 상태 불일치나 허용 시간을 넘는 지연은 제품 결함으로 조사하겠습니다.

문제4: AI 생성 코드 검증 자동화 (코드 필수)

과제

- AI/LLM을 활용해 QA 자동화 생산성을 높일 수 있는 도구 아이디어를 1가지 제안하고, 동작 구조를 설계해주세요. (예: 기획서 기반 TC 자동 생성, 실패 로그 자동 분류, Self-healing 셀렉터 등)
- 프로토타입 코드나 프롬프트 설계를 함께 제출하면 가점이 있습니다.

문제 4 답변

기존에 테스트 자동화를 구축하면서 가장 많은 시간이 들었던 작업은 테스트 스크립트 작성이었습니다. 화면 요소를 찾고 같은 동작과 검증 코드를 반복해서 작성하는 시간을 줄이고자 Universal 프로젝트를 만들기 시작했습니다. 이번 과제를 받고 기존 기능에 AI 지원을 추가한 Android Script Studio를 제출합니다.

휴대폰 화면에서 수행한 동작을 기록해 Python 또는 Kotlin 테스트 코드로 저장하는 도구입니다. 추가한 AI 기능은 테스트 목표와 현재 화면을 보고 다음 동작을 제안합니다. 실행한 동작을 기록하고, 기록을 바탕으로 테스트 코드를 생성합니다.

프로토타입 소스 코드와 프롬프트, [Windows EXE 플 패키지](#)를 함께 제출합니다. 실행 및 설정 방법은 [README](#)에서 확인할 수 있습니다.

상세 설명

Q1. 어떤 기능을 제공하고, AI는 무엇을 맡는가?

A. 기존 화면 조작 기록과 코드 저장 기능에, 목표에 따라 다음 동작을 추천하거나 자동으로 진행하는 AI 기능을 추가했습니다.

- 수동 작성:** 휴대폰 화면의 요소를 선택해 클릭, 텍스트 입력, 표시 확인, 뒤로가기, 스크롤, 대기를 기록
- AI 지원:** 목표, 도착 문구와 현재 화면을 바탕으로 다음 동작 제안. 추천만 받거나, 실행 조건 확인 후 자동 진행
- 코드 저장:** 완료된 동작과 검증을 Python(Appium) 또는 Kotlin(UiAutomator)으로 저장하고 시작 조건과 단계 기록도 함께 보관
- 사용량 확인:** AI 호출의 토큰 사용량과 추정 비용 표시

AI는 임의의 실행 코드를 작성해 바로 실행하지 않습니다. 다음 동작만 정해진 형식으로 제안하고, 코드 생성은 프로그램의 템플릿을 사용합니다. QA는 시작 상태와 기대 결과를 정하고 생성된 코드를 검토합니다.

Q2. 동작 구조와 결과 검증은 어떻게 설계했는가?

A. 목표 입력 → 화면 확인 → AI 제안 → 대상 확인 및 실행 → 결과 기록 → 코드 저장 순서로 구성했습니다.

1. **준비:** 사용자가 테스트 앱의 시작 상태, 목표와 도착 문구를 설정
2. **화면 확인:** 현재 화면의 버튼과 문구 등 접근성 정보를 수집하고 선택 가능한 요소만 AI에 전달
3. **제안 검증:** 존재하지 않는 대상이나 잘못된 응답 형식은 거절. 화면 변경이나 중복 대상은 실행 중단
4. **실행과 재확인:** 한 동작을 실행한 뒤 화면을 다시 확인. 도착 문구가 실제로 보여야 표시 검증을 기록
5. **저장:** 실제 완료한 동작만 코드에 반영하고 Python/Kotlin과 시나리오 파일 생성

주문, 결제, 동의와 권한 관련 문구는 자동 진행을 멈추고 사용자가 판단하도록 했습니다. 이미지 전용 화면과 비밀번호 입력은 지원하지 않습니다. 화면에 문구가 보이는 것만으로 주문 성공을 판단하지 않으며, 주문 ID와 금액 같은 업무 검증은 별도로 추가해야 합니다.

AI에는 목표와 일부 화면 요소의 문구/속성을 보내므로 테스트 데이터를 사용합니다. 전체 XML과 스크린샷은 보내지 않습니다.

Q3. 어떻게 실행하고 설정하는가?

A. Windows x64용 ZIP 전체를 풀고 AndroidScriptStudio.exe를 실행합니다. Python, Java, Node.js, ADB, Appium과 UiAutomator2를 함께 넣어 작성기 실행에 필요한 개발 도구를 별도로 설치하지 않도록 구성했습니다.

1. **압축 해제:** 쓰기 가능한 짧은 경로에 ZIP 전체를 해제. EXE만 따로 옮기지 않음
2. **단말 준비:** Android 휴대폰의 USB 디버깅 활성화, USB 연결과 PC 승인. 인식되지 않으면 제조사 USB 드라이버 설치
3. **AI 설정:** 프로그램에서 사용할 서비스, 본인의 API 키와 모델 설정. AI 호출에는 인터넷과 별도 사용료가 필요
4. **사용:** 휴대폰 연결 후 목표와 도착 문구 입력 → 수동 또는 AI 지원으로 작성 → 기록 검토 후 코드 저장

동봉한 **환경진단.cmd**로 실행 도구와 프로그램 창, Appium 서버를 확인할 수 있습니다. 개인 API 키와 단말 기록은 배포하지 않습니다. 내보낸 테스트를 재생할 Python 환경이나 Kotlin용 Android Studio 테스트 프로젝트는 별도로 준비해야 합니다.